EE671: FOUNDATION OF VLSI CAD

ASSIGNMENT – 1

NAME : Dharavath Nikhitha

Roll Number : 26M1167

1. Introduction

This assignment implements a simple combinational logic simulator in Python for a structural Verilog netlist. The simulator parses the circuit, identifies its inputs and outputs, evaluates the logic gates, and produces the corresponding output for given input vectors. A 1-bit full adder implemented using 2-input NAND gates is used to demonstrate and verify the simulator.

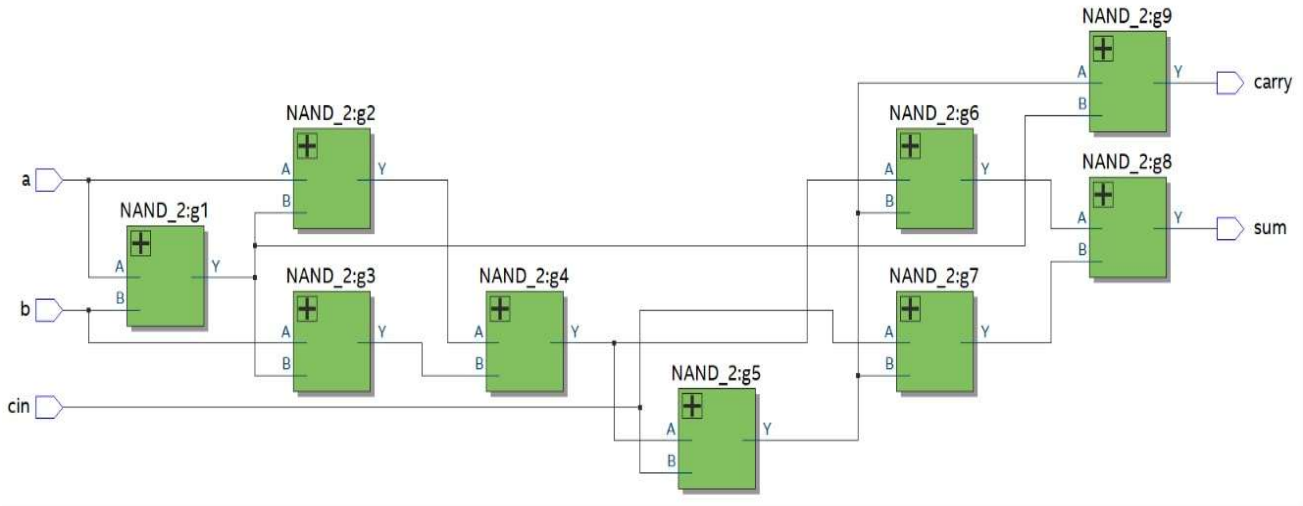
.2. Objectives

- Parse a structural Verilog netlist using Python.
- Identify inputs, outputs, and internal signals.
- Evaluate the logic gates in the correct order.
- Simulate the circuit for given input vectors.
- Verify the full-adder operation for all possible input combinations.
- Generate the corresponding simulation results.

.3. Files and Inputs Used

File	Purpose	Key contents
logic_simulator.py	Python logic simulator	Verilog parser, graph construction, levelization, gate evaluation, CLI and batch simulation
fulladder.v	Structural Verilog netlist	1-bit full adder using 9 NAND gates
input_vectors.txt	Batch input vectors	000, 001, 010, 011, 100, 101, 110, 111
output_results.txt	Simulation output	Input vector mapped to sum/carry output

4. Full-Adder Gate-Level Circuit



4.1 Gate-Level Netlist

```
nand g1 (w1, a, b);

nand g2 (w2, a, w1);

nand g3 (w3, b, w1);

nand g4 (w4, w2, w3);

nand g5 (w5, w4, cin);

nand g6 (w6, w4, w5);

nand g7 (w7, cin, w5);

nand g8 (sum, w6, w7);

nand g9 (carry, w5, w1);
```

4.2 Logic Operation

$w1 = \sim(a \cdot b)$

$w2 = \sim(a \cdot w1)$

$w3 = \sim(b \cdot w1)$

$w4 = \sim(w2 \cdot w3) = a \oplus b$

$w5 = \sim(w4 \cdot cin)$

$w6 = \sim(w4 \cdot w5) = w4 \oplus cin$

$w7 = \sim(cin \cdot w5)$

$sum = \sim(w6 \cdot w7)$

$carry = \sim(w5 \cdot w1)$

5. Python Logic Simulator Implementation

The submitted Python program is organized around a Gate class and a sequence of parsing, graph-building, levelization, and simulation functions. The implementation actually supplied in the ZIP file is used as the basis of this report.

5.1 Gate Representation and Evaluation

Each Gate object models a logic gate. Supports logic evaluation for:

- **Basic gates**: AND, OR, NOT, NAND, NOR, XOR, XNOR
- **Signal handlers**: INPUT, WIRE, BUF

5.2 Verilog Parsing

- Parses module declarations, input/output ports, wires, and gate instances from the Verilog netlist.
- Identifies gate type, gate name, output signal, and input signals.
- Builds the circuit structure for logic evaluation and simulation.

5.3 Dependency Graph Construction

The simulator creates a driver map that records which gate produces each signal. It then builds dependency and dependent sets. If a gate uses an internal signal produced by another gate, that producing gate becomes a dependency of the current gate.

5.4 Levelization

The logic level of each gate is calculated based on the levels of its input signals.

Formula: Level = max(input levels) + 1

1. Input signals are assigned Level 0.
2. For each gate, the maximum input level is identified.
3. The gate level is then calculated by adding 1.
4. This ensures that gates are evaluated in the correct order during simulation.

5.5 Simulation and Batch Processing

For each input vector, the simulator assigns the bits to the primary inputs, evaluates gates in levelized order, checks that the primary outputs have been calculated, and concatenates the outputs in primary-output order. In batch mode, all vectors are read from input_vectors.txt and the corresponding results are written to output_results.txt.

5.6 Command-Line Usage

python logic_simulator.py fulladder.v input_vectors.txt output_results.txt

6. Gate Information and Levelization

The simulator identified three primary inputs, two primary outputs, and nine gates.

Gate	Type	Output	Inputs
g1	NAND	w1	a, b
g2	NAND	w2	a, w1
g3	NAND	w3	b, w1
g4	NAND	w4	w2, w3
g5	NAND	w5	w4, cin
g6	NAND	w6	w4, w5
g7	NAND	w7	cin, w5
g8	NAND	sum	w6, w7
g9	NAND	carry	w5, w1

6.1 Levelized Order

Level	Gate(s)
Level 1	g1
Level 2	g2, g3
Level 3	g4
Level 4	g5
Level 5	g6, g7, g9
Level 6	g8

7. Simulation Setup and Results

Input (a b cin)	Expected sum	Expected carry	Simulator output (sum carry)
000	0	0	00
001	1	0	10
010	1	0	10
011	0	1	01
100	1	0	10
101	0	1	01
110	0	1	01
111	1	1	11

```
PS C:\Users\nikhi> cd "C:\Users\nikhi\Documents"
PS C:\Users\nikhi\Documents> python logic_simulator.py fulladder.v input_vectors.txt output_results.txt



======================================================================
LOGIC SIMULATOR
======================================================================
Primary Inputs : a, b, cin
Primary Outputs: sum, carry
Number of Gates: 9


======================================================================
GATE INFORMATION
======================================================================
g1   : NAND  Output = w1      Inputs = a, b
g2   : NAND  Output = w2      Inputs = a, w1
g3   : NAND  Output = w3      Inputs = b, w1
g4   : NAND  Output = w4      Inputs = w2, w3
g5   : NAND  Output = w5      Inputs = w4, cin
g6   : NAND  Output = w6      Inputs = w4, w5
g7   : NAND  Output = w7      Inputs = cin, w5
g9   : NAND  Output = carry   Inputs = w5, w1
g8   : NAND  Output = sum     Inputs = w6, w7


======================================================================
GATE LEVELIZATION
======================================================================
Level 1: g1
Level 2: g2, g3
Level 3: g4
Level 4: g5
Level 5: g6, g7, g9
Level 6: g8
======================================================================
```

```
======================================================================
SIMULATION RESULTS
======================================================================
000 --> 00
001 --> 10
010 --> 10
011 --> 01
100 --> 10
101 --> 01
110 --> 01
111 --> 11
======================================================================

Results successfully saved to:
output_results.txt
PS C:\Users\nikhi\Documents>
```

.

8. Observations and Discussion

- The parser identified 3 primary inputs (a, b, cin) and 2 primary outputs (sum, carry).
- The simulator detected 9 NAND gates, matching the structural Verilog design.
- The circuit was divided into 6 evaluation levels based on gate dependencies.
- All 8 possible input combinations were simulated successfully.
- The obtained outputs matched the expected 1-bit full-adder truth table.
- The simulation results were successfully saved to output_results.txt.
- The simulator supports multiple combinational gate types, while the demonstrated circuit uses only NAND gates.

9. Limitations and Possible Improvements

- The parser supports simplified structural Verilog rather than the complete Verilog language.
- Only combinational circuits are currently supported.
- The simulator uses binary 0/1 logic and does not support X and Z states.
- Timing, delay, power, and waveform analysis are not included.
- Support for sequential circuits, buses, and additional Verilog constructs can be added.
- A GUI and waveform viewer could improve usability.

10. Conclusion

A Python-based structural logic simulator was successfully developed and tested using a 1-bit full adder implemented with nine NAND gates. The simulator parses the Verilog netlist, analyzes gate dependencies, assigns evaluation levels, and performs gate-level simulation. All eight input combinations produced the expected full-adder outputs, confirming the correctness of the implementation.